

Dokumentation: Virtuelles Vereinsfondsverwaltungssystem

1. Übersicht

Das virtuelle Vereinsfondsverwaltungssystem ermöglicht die Verwaltung von Konten, Benutzern und finanziellen Transaktionen. Die Hauptfunktionen umfassen die Benutzerverwaltung, Kontoverwaltung, Transaktionen sowie Finanzberichte.

2. Systemarchitektur

Das Programm folgt einer modularen Architektur mit klar getrennten Verantwortlichkeiten für Benutzeroberfläche (GUI), Controller, Modelle und Datenhaltung.

- **GUI-Komponenten:** AdminView, KassenwartView, FinanceView
- **Controller:** AuthManager, CSVManager
- **Modelle:** User, Account

Die Komponenten arbeiten zusammen, um Benutzereingaben zu verarbeiten, Daten zu speichern und verschiedene Benutzerrollen mit spezifischen Zugriffsrechten zu unterstützen.

3. UML-Diagramme

3.1 UML-Klassendiagramm

Das Klassendiagramm beschreibt die wichtigsten Klassen im System und ihre Beziehungen.

- **User:** Speichert Benutzerinformationen wie Benutzernamen, Passwort und Rolle.
- **Account:** Enthält Kontodaten wie Kontonummer, Abteilung und Kontostand.
- **CSVManager:** Verwaltet das Laden und Speichern von Benutzerdaten und Kontodaten aus CSV-Dateien.
- **AuthManager:** Verifiziert Benutzerinformationen und authentifiziert Benutzer.
- **GUI-Klassen:** Repräsentieren die Benutzeroberflächen für verschiedene Benutzerrollen.

3.2 UML-Zustandsdiagramm

Das Zustandsdiagramm zeigt den Ablauf des Programms:

1. Initialer Zustand: Programmstart.
2. Login-Seite: Benutzer geben Benutzernamen und Passwort ein.
3. Authentifizierung über `AuthManager`.
4. Erfolgreiche Anmeldung: Benutzer werden basierend auf ihrer Rolle weitergeleitet:
 - `AdminView`
 - `KassenwartView`
 - `FinanceView`
5. Fehlerhafte Anmeldung: Rückkehr zur Login-Seite.

3.3 UML-Anwendungsfalldiagramm

Das Anwendungsfalldiagramm beschreibt die Hauptfunktionen des Systems:

- **Admin:**
 - Konto erstellen (`create_account()`)
 - Konto zuweisen (`assign_account()`)
 - **Kassenwart:**
 - Einzahlen (`deposit()`)
 - Abheben (`withdraw()`)
 - Überweisen (`transfer()`)
 - **Finance:**
 - Kontostände anzeigen (`view_balance()`)
 - Finanzübersicht anzeigen (`view_summary()`)
-

4. Datenstruktur

4.1 Benutzerklasse (`User`)

Attribute:

- `username: str`
- `password: str`
- `role: str`

4.2 Kontoklasse (`Account`)

Attribute:

- `account_id: str`
- `department: str`
- `balance: float`

4.3 CSV-Datenhaltung

Benutzerdaten und Kontodaten werden in CSV-Dateien gespeichert und beim Programmstart geladen.

- Datei: `users.csv`
 - Datei: `accounts.csv`
-

5. Funktionen und Methoden

5.1 Authentifizierung

- `AuthManager.authenticate(username, password)`: Verifiziert die Benutzeranmeldeinformationen und gibt bei Erfolg ein Benutzerobjekt zurück.

5.2 Datenmanagement

- `CSVManager.load_users()`: Lädt Benutzerdaten aus der CSV-Datei.
- `CSVManager.save_users(users)`: Speichert Benutzerdaten in der CSV-Datei.
- `CSVManager.load_accounts()`: Lädt Kontodaten.
- `CSVManager.save_accounts(accounts)`: Speichert Kontodaten.

5.3 Benutzeroberfläche

- **AdminView**: Ermöglicht das Erstellen und Zuweisen von Konten.
 - **KassenwartView**: Ermöglicht Einzahlungen, Abhebungen und Überweisungen.
 - **FinanceView**: Zeigt Kontostände und Finanzübersicht an.
-

6. Laufzeitszenarien

6.1 Benutzeranmeldung (Admin)

1. Benutzer startet das Programm.
2. Auf der Login-Seite gibt der Benutzer seine Zugangsdaten ein.
3. `AuthManager` prüft die Zugangsdaten:
 - Bei Erfolg: Weiterleitung zur `AdminView`.
 - Bei Misserfolg: Rückkehr zur Login-Seite.

6.2 Einzahlen (Kassenwart)

1. Kassenwart meldet sich an und wird zur `KassenwartView` weitergeleitet.
2. Wählt die Option "Einzahlen".
3. Gibt den Betrag und das Zielkonto ein.
4. Betrag wird dem Konto gutgeschrieben und in der CSV-Datei gespeichert.

7. Anforderungen und Abhängigkeiten

- **Python-Version:** 3.x
- **Benötigte Module:**
 - `tkinter` für die GUI
 - `csv` für die Datenverwaltung

8. Fehlerbehandlung und Stabilität

- Eingaben werden auf Korrektheit überprüft (z.B. leere Eingabefelder).
- Fehlerhafte Benutzeranmeldungen werden abgefangen und an den Benutzer zurückgemeldet.
- Kontostände werden geprüft, um negative Salden zu vermeiden.

9. Weiterentwicklung

Mögliche Erweiterungen:

- Datenbankunterstützung anstelle von CSV-Dateien.
- Erweiterte Benutzerrechteverwaltung.
- Zusätzliche Berichtsfunktionen.

10. Fazit

Das virtuelle Vereinsfondsverwaltungssystem bietet eine stabile Lösung für die Verwaltung von Vereinsfinanzen mit einer benutzerfreundlichen Oberfläche und klar strukturiertem Code. Die Systemarchitektur ermöglicht zukünftige Erweiterungen und Anpassungen.